

Sharif University of Technology

Department of Mathematical Sciences

A Survey on Foundational Concepts of Zero-Knowledge Proofs

Submitted by:
Nadia Afsar

Submission Date: October 2, 2026

Abstract

This report surveys the fundamentals of zero-knowledge proofs (ZKPs). It first presents the formal definitions of completeness, soundness, and zero-knowledge, and then examines classic examples such as Graph 3-Coloring and the Schnorr protocol. It discusses techniques for reducing the soundness error, including sequential and parallel composition, and reviews non-interactive constructions via the Fiat–Shamir transform and modern systems like STARKs. The survey concludes that ZKPs are a powerful cryptographic tool, but their security still relies on computational assumptions and their efficiency can be further improved.

Table of Contents

Introduction	1
Preliminaries and Definitions	1
A Classic Example: The Graph 3-Coloring Protocol	4
A Classic Example of ZKPoK: The Quadratic Residuosity Problem	5
Schnorr’s Protocol	5
Improving Efficiency: Reducing the Soundness Error	6
Constant-Round Zero-Knowledge Proof	7
Non-Interactive Zero-Knowledge Proofs	7
ZK-STARK	8
Conclusion	9
References	10

Introduction

In the spy game, in each round a word is given to everyone except the spy. People ask each other questions about the word, and each person is required to convince the person who has asked them that they know the word. On the other hand, they should try not to let the spy guess the word, so their answers should yield as little information as possible. This scenario illustrates the core idea behind zero-knowledge proofs.

Due to the growth of digital communication, ensuring the privacy of transferred data has become a serious challenge. Cryptography offers mathematical tools to solve this issue, among which Zero-Knowledge Proofs have gained significant popularity recently. Zero-Knowledge Proofs allow a prover to convince a verifier of a statement without revealing any additional information. Such protocols can be used in many areas, which is why they have become such a interesting topic in modern cryptography.

As mentioned, Zero-Knowledge Proofs have a wide variety of applications. For instance, they are used in **authentication systems** and **identification protocols**. They were also applied in the Zerocoin and Zerocash protocols, contributing to the development of **blockchain technology**. In the context of **electronic voting**, ZKPs allow voters to prove their eligibility and cast their votes without revealing any information about their identity or their actual vote.

This is a survey of some personal studies on the foundations of Zero-Knowledge Proofs. First, we will examine different notions and relaxations of ZKP, including their formal definitions and classic examples such as Graph 3-Coloring, Quadratic Residuosity, and the Schnorr protocol. We will then discuss techniques for improving protocol efficiency while minimizing completeness and soundness errors. Finally, we will review some existing protocols for these applications, covering non-interactive zero-knowledge proofs and practical systems like STARKs.

Preliminaries and Definitions

Cryptographic Proof Systems

First we need to define what we mean by proofs. Proofs are a set of statements that convince you that a claim is true. Each proof consists of two parties: the **prover** that provides the proof and the **verifier** that ensures the given proof is valid. In cryptography, these proof protocols are evaluated by three properties: **Completeness**, **Soundness**, and **Zero-Knowledge**. The **Completeness** property causes the verifier to be convinced provided that the statement is true. The **Soundness** property asserts that the verifier rejects invalid statements. The **Zero-Knowledge** property ensures that the verifier does not gain any knowledge beyond the truth of the claim, and by gaining knowledge, we mean gaining knowledge that the verifier

could not gain by itself.

If the soundness property holds when the prover runs in polynomial time, it is said that the system has the **computational** soundness property and is called an **argument** instead of a proof.

Interactive Proof Systems

A common approach for designing such protocols is through **interactive systems**, where the prover exchanges messages with the verifier to prove its claim. A pair of interactive machines (P, V) is called an interactive proof system for language L if the completeness and soundness properties hold with high probability:

Completeness: For every $x \in L$, $\Pr[(P, V)(x) = 1] \geq \frac{2}{3}$,

Soundness: For every $x \notin L$ and every interactive machine B , $\Pr[(B, V)(x) = 1] \leq \frac{1}{3}$.

If these systems come with the zero-knowledge property, they are said to be **zero-knowledge proofs (ZKP)**.

Zero-Knowledge Proofs of Knowledge

One type of ZKP is the **Zero-Knowledge Proof of Knowledge (ZKPoK)**, in which the prover aims to prove that it knows a **witness** for a given statement without revealing any information about the witness itself. The properties for these proofs are redesigned accordingly. The proof system is said to have the completeness property if the verifier accepts the proof provided that the prover indeed knows the witness. Knowledge soundness is a security property that guarantees that if the verifier accepts the proof, then the prover must actually know the witness. This property is typically proved by demonstrating the existence of an **extractor** that, by interacting with the prover, can output a valid witness for the statement.

Honest-Verifier Zero-Knowledge

A weaker but useful notion is that of **Honest-Verifier Zero-Knowledge (HVZK)**. In HVZK, we only require that the view of the verifier can be simulated when the verifier follows the protocol honestly. This is different from standard zero-knowledge, where the simulator must work for any possible verifier, even malicious ones.

Σ -protocols

A well-known class of HVZKPs is that of **Σ -protocols**, which are constructed for proofs of knowledge. Sigma protocols work as follows: The common input is a statement x , and the prover knows a witness w for it. First, the prover sends an **announcement** a . Then the verifier responds with a random **challenge** c . Finally, the prover sends a **response** r . The verifier then checks whether the triple (a, c, r) satisfies a predefined relation, which confirms that the prover indeed knows the witness.

Non-Interactive Zero-Knowledge Proof

If the prover sends a single message, and the verifier validates the claim based on that message without any further interaction, the system is called a **Non-Interactive Zero-Knowledge Proof (NIZKP)**. These proofs are typically realized using a **common reference string (CRS)** model.

Levels of Zero-Knowledge: Perfect, Statistical, and Computational

For proving the zero-knowledge property, a **simulator** is usually designed to simulate the view of the verifier such that it cannot be distinguished from the real messages the verifier receives from the prover. However, in practice, achieving **perfect zero-knowledge**—which requires that the simulated and real views have identical probability distributions—is often not feasible. Therefore, two new concepts are defined:

A proof protocol is said to be **statistically zero-knowledge** if a simulator can be found such that the probability of distinguishing between the simulated view and the real view is negligible.

A proof protocol is said to be **computationally zero-knowledge** if a simulator can be found such that no polynomial-time distinguisher can distinguish between the simulated view and the real view with more than negligible probability.

Auxiliary-Input Zero-Knowledge

In real-world applications, the verifier often possesses a priori information, denoted z , beyond the common input x . This auxiliary input might help the verifier extract knowledge that the basic definition fails to protect against. Hence, we adopt the stronger, standard notion of **auxiliary-input zero-knowledge**. A proof system satisfies this definition if the zero-knowledge properties hold for every possible auxiliary input z given to the verifier—even if z is correlated with x —and, crucially, the simulator is also given z to account for this extra information.

Witness Indistinguishability and Independence

Another relaxation of the zero-knowledge property is witness-indistinguishability. If a statement has several possible witnesses, it is said to be **witness-indistinguishable** if the views of the verifier resulting from interactions using any two witnesses are computationally indistinguishable. If these views are statistically indistinguishable, the protocol is said to be **witness-independent**.

One-Way Functions

One-way functions are functions that are easy to compute, but finding a preimage for a given output is hard for any efficient algorithm. It's essential to assume that one-way functions exist for several foundations of cryptography, and constructing **commitment schemes** is a key example of what becomes possible under this assumption.

Commitment Schemes

A commitment involves two parties: the **sender** and the **receiver**. The process runs as follows. First, the sender commits to a message and sends the resulting commitment to the receiver. Later, in the reveal phase, the sender reveals the original message. The property that ensures the sender cannot later reveal any message other than the original one is called **binding**. The **hiding** property guarantees that the receiver cannot determine what the message is during the commit phase.

Assuming f is a 1-1 one-way function and $b : \{0, 1\}^* \rightarrow \{0, 1\}$ is a hard-core predicate, a construction for commitment schemes is as follows: In the commit phase, the sender sends $(f(s), b(s) \oplus v)$ as a commitment to v , where s is uniformly selected from $\{0, 1\}^n$. Then, in the reveal phase, the sender reveals v and s , and the receiver accepts if $f(s) = \alpha$ and $b(s) \oplus v = \sigma$, where (α, σ) is the received message in the commit phase from the sender. If the sender tries to reveal a different bit instead of v while still satisfying $b(s) \oplus v = \sigma$, they would need to provide another s' such that $f(s') = \alpha$. But since f is 1-1, the only preimage of α is the original s , so they cannot change the bit. This proves the binding property. For hiding, because s is random and b is a hard-core predicate, no efficient algorithm can distinguish $b(s)$ from a random bit when given only $f(s)$. Hence, commitments to 0 and 1 are computationally indistinguishable, ensuring the hiding property.

A Classic Example: The Graph 3-Coloring Protocol

Most of the problems desired to be solved via ZKP are NP problems. So we'd like to find ZKPs for every NP problem and for that all we have to do is proving ZKP exist for a NP-complete problem. A famous one is Graph 3-coloring problem. The proof protocol runs as follow:

The common input is the graph $G = (V, E)$. The prover chooses a random permutation π over $\{1, 2, 3\}$ and sets $\phi(v) = \pi(\psi(v))$ for each $v \in V$ that ψ is the coloring the prover knows. Then the prover sends commitments to these $\phi(v)$ values and sends them to the verifier.

The verifier chooses a random edge and asks the prover to reveal the color of its vertices and accepts the proof if their colors are different.

We notice that if ψ is a true coloring, then in each permutation the color of neighbors are different so completeness holds. And if the prover doesn't know any true coloring there exists at least one edge with same color vertices and makes the proof to fail so the soundness holds with the probability at least $1/|E|$. However this isn't a good soundness error, we'll see some approaches for reducing the error later.

For proving the zero-knowledge we provide a simulation as follows: the simulator commits to a random coloring and if the edge wanted in the verifier's challenge has vertices with different colors in the random coloring, outputs the colors, otherwise halts. So if the simulator didn't halt it succeed in simulating the view of the verifier. It's easy to see the simulator halts with probability $1/3$ for an honest verifier, and for malicious verifiers the hiding property of the commitment scheme bounds the failure by at most $1/2$ which is acceptable for our purposes.

Previously, we proved that assuming one-way functions exist, we can construct a commitment scheme with hiding and binding properties. So with that assumption, we're close to proving that $\mathcal{NP}$ problems

have ZKPs.

A Classic Example of ZKPoK: The Quadratic Residuosity Problem

It is important to note that not all zero-knowledge proofs are guaranteed to satisfy knowledge soundness. Here is a famous and classic example of a zero-knowledge proof of knowledge, constructed for the quadratic residuosity problem.

The common input is $x \in QR_N$. The prover knows a witness w such that $x \equiv w^2 \pmod{N}$. First, the prover sends $y = r^2$ to the verifier, where r is chosen uniformly at random from $\mathbb{Z}_N^*$. Then, the verifier sends a random bit $b \in \{0, 1\}$. Next, the prover responds with $z = r \cdot w^b$. The verifier accepts if $z^2 \equiv y \cdot x^b \pmod{N}$.

It is clear that if the prover knows a valid witness, they always succeed, proving completeness. To prove knowledge soundness, we introduce a knowledge extractor K . First, K invokes the prover and receives y . Then, K sends the query $b = 0$ and receives $z_0 = r$. After that, K rewinds the prover and sends the query $b = 1$, receiving $z_1 = w \cdot r$. Finally, it outputs $z_1/z_0 = w$ as the witness.

To prove the zero-knowledge property, we provide a simulator. For an honest verifier, the simulator simply chooses random z and b , and outputs $(z^2/x^b, b, z)$ as the verifier's view. For a cheating verifier, the simulator repeatedly chooses random z and b until the verifier's query equals the chosen b , at which point it outputs $(z^2/x^b, b, z)$. The probability that the simulator's chosen b matches the verifier's query is $1/2$, so the expected number of repetitions is 2.

Schnorr's Protocol

A famous Σ -protocol is Schnorr's protocol, which is constructed for the discrete logarithm problem. Suppose the common input is an element h in a group $\mathbb{G}$ of prime order n , with a known generator g . The prover knows a witness $x \in \mathbb{Z}_n$ such that $h = g^x$, and tries to convince the verifier without revealing x . The protocol works as follows:

First, the prover chooses u randomly from $\mathbb{Z}_n$ and sends the announcement $a = g^u$ to the verifier. Then the verifier sends the challenge c , chosen randomly from $\mathbb{Z}_n$. The prover generates the response $r = u + cx \pmod{n}$ and sends it to the verifier. Finally, the verifier checks if $g^r = ah^c$.

If the prover knows a correct witness x such that $h = g^x$, then $g^r = g^{u+cx} = g^u(g^x)^c = ah^c$. So completeness holds.

In addition, if the prover can produce two accepting transcripts with the same announcement a but different challenges $c \neq c'$ and responses r, r' , then $g^r = ah^c$ and $g^{r'} = ah^{c'}$. Dividing these equations

gives $g^{r-r'} = h^{c-c'} = g^{x(c-c')}$, so $x = (r - r') / (c - c') \pmod n$. Thus, an extractor can easily extract the witness x from the prover, and knowledge soundness holds.

For proving zero-knowledge, we define a simulator as follows: it first chooses random c and r , and computes $a = g^r h^{-c}$ as the announcement. For any challenge c , the equality $g^r = ah^c$ holds, and the simulator successfully simulates the view of the verifier.

Improving Efficiency: Reducing the Soundness Error

As it was mentioned, the soundness error of the interactive proof for the 3-coloring problem is not acceptable and needs to be reduced. In algorithm analysis, the most straightforward approach to reducing the error of a randomized protocol is repetition. There are two types of composition: sequential and parallel.

Reducing the Soundness Error via Sequential Composition

We saw the soundness error of the proof constructed for the 3-coloring problem is at most $1 - 1/|E|$, which is not negligible. By repeating the protocol k times sequentially, this error reduces to $(1 - 1/|E|)^k$, which becomes negligible for sufficiently large k . When it comes to preserving zero-knowledge, sequential composition works perfectly for the auxiliary-input definition of ZK. The reason is simple: the simulator for each round is given the entire transcript of the previous rounds as its auxiliary input, so it can simulate the next round conditioned on the verifier's current state. However, the basic (non-auxiliary) computational zero-knowledge definition is not closed under sequential composition because the simulator is not given this accumulated transcript, and thus cannot account for the verifier's acquired knowledge between rounds. For witness indistinguishability and witness independence, sequential composition holds as well, since these properties only require that the view remains indistinguishable regardless of which witness is used; a hybrid argument over the sequential rounds proves this. Finally, the zero-knowledge property for proofs of knowledge is also preserved through sequential repetitions, which we will discuss in more detail later.

Reducing the Soundness Error via Parallel Composition

Parallel composition also reduces the soundness error exponentially (similarly to sequential), and it is often preferred in practice because it saves time. Unfortunately, zero-knowledge is not generally preserved under parallel executions, because a cheating verifier may be able to extract information through parallel interactions that it couldn't gain on their own. For example, imagine a prover P_1 that reveals a secret only if the verifier answers a random hard question correctly, and another prover P_2 that answers such questions. Each prover individually is zero-knowledge, but running them in parallel lets the verifier forward P_1 's question to P_2 and use the answer to extract the secret from P_1 . However, there exist some protocols that can be improved with parallel composition and still remain zero-knowledge. The good news is that witness-indistinguishability (and its statistical variant, witness-independence) are preserved

under parallel composition, which can be proved using a hybrid argument.

Constant-Round Zero-Knowledge Proof

We saw that soundness error can be reduced through both sequential and parallel composition. However, we are interested in protocols that are efficient with respect to both time and error probability. In other words, we would like to find constant-round protocols with negligible error probability. In the graph 3-coloring example, the main issue was its non-negligible soundness error, which we addressed by performing sequential executions. While this works for reducing the error, the number of rounds grows, making it inefficient. Parallel execution successfully reduces the soundness error in a constant number of rounds, but unfortunately, it does not preserve the zero-knowledge property.

One might consider changing the order of steps so that the verifier sends the edge queries first, and then the rest of the protocol proceeds normally. However, this naive approach would let the prover cheat, and soundness would completely fail. To improve this, we can modify the protocol so that the verifier first sends commitments to the edges for all parallel executions. After the prover sends its commitments to the colorings, the verifier reveals the committed edges, and the rest of the protocol proceeds as before. This way, a cheating verifier cannot adaptively choose the edges based on the prover's messages, which preserves the zero-knowledge property.

This idea leads to constant-round zero-knowledge proofs, provided we use a perfectly hiding commitment scheme. The simulator for such protocols works by first extracting the verifier's committed edges and then generating a transcript that is computationally indistinguishable from a real one. The key point is that the simulator may need to rewind the verifier a few times, so its running time is expected polynomial time, not strict. However, this construction typically requires stronger assumptions than standard one-way functions, such as the existence of claw-free permutations. Still, it shows that constant-round ZKPs are achievable in principle, which is an important theoretical and practical step toward efficient protocols.

Non-Interactive Zero-Knowledge Proofs

There are cases when the prover isn't supposed to convince a specific verifier and needs to provide a proof which any verifier can verify. That's why we need non-interactive zero-knowledge proof protocols.

A Non-Interactive Zero-Knowledge Proof (NIZK) is a ZKP where the prover sends a single message to the verifier, and the verifier checks it without any back-and-forth interaction. This is useful in settings like blockchain or public bulletin boards, where the prover cannot wait for a response from a specific verifier, and the proof needs to be verified by anyone at any time. However, removing interaction creates a problem: in interactive proofs, the verifier's random challenges prevent the prover from cheating. To replace these challenges, NIZKs typically rely on a Common Reference String (CRS), a random string generated by a trusted party and made available to both the prover and all verifiers.

The Hidden-Bits Model

One abstraction that helps in the construction of non-interactive proofs is the hidden-bit model. In this model, the prover reveals a subset of the bits from a common random string that are needed for the proof, while the verifier only sees the revealed bits, not the entire string. Using this model, non-interactive zero-knowledge proof systems can be constructed for NP-complete problems, such as the Hamiltonian cycle problem. However, this model is not efficient in practice because the required CRS is too large, and the proof protocol is restricted to revealing only specific bits rather than having full access to the random string.

The Fiat-Shamir Transform

An interesting approach is the Fiat-Shamir Transform, which transforms a public-coin (where the messages sent by the verifier are uniform random strings) interactive argument into a non-interactive argument. The idea of the transformation is as follows: Imagine an interactive argument between a prover and a verifier where x is the common input, α_i 's are the messages sent by the prover, and β_i 's are the messages sent by the verifier. The prover needs to somehow generate the β_i 's so it can send all α_i 's at once, and then there's no need for interaction. We can do such a thing through hash functions. Assume H is a secure hash function. Then the prover generates β_i 's this way: $\beta_i = H(x, \alpha_1, \dots, \alpha_i)$. The security of the resulting argument depends on the assumption we make about the hash function. The bad news is that we still don't know such a hash function. For the rest of this part, we assume such a hash function exists.

A key advantage of the Fiat-Shamir transform is that it does not require any trusted setup, unlike many other NIZK constructions that rely on a Common Reference String (CRS) generated by a trusted party.

It's easy to see that if the prover knows a correct witness, it can successfully generate the β_i 's and convince the verifier, assuming the original interactive protocol has completeness. It's important to notice that soundness is not preserved through this transform for all multi-round protocols. But the soundness of the transformed protocol holds for every constant-round protocol. For the soundness property, it can be proved that if soundness doesn't hold for the constructed non-interactive protocol, then we can break the soundness of the original protocol. For proving the zero-knowledge property, the simulator chooses the transcript randomly and succeeds in simulating the verifier's view because, in the random oracle model, the string generated by the hash function is indistinguishable from a truly random string.

However, constructing security protocols based on heuristic assumptions is dangerous, yet this transformation is widely used in practice.

ZK-STARK

In blockchain, we need inclusive accountability with which everyone can verify the integrity of the system using their own devices, instead of relying on centralized accountability. This way, two problems come up. The first problem is that when everyone can verify, reaching privacy becomes more difficult. The other problem is scalability. We need high throughput, but the verification must be cheap and fast so it can be done on weaker devices, too. ZK-STARKs are a family of ZKPs which accomplish this need.

These systems must have four properties that explain why they are called ZK-STARK. First, they need to be **Zero-Knowledge**, which is defined before. They also need to have **Scalability**, which means for computations lasting T cycles, proofs should be generated in roughly T cycles (quasi-linear in T) and verified exponentially faster than T (about $\log T$ cycles). In addition, they need to be **Transparent**, or in other words, public-coin. Finally, they need to be an **Argument of Knowledge**, which ensures that the proof can only be generated by a prover who knows a private input (witness).

To provide a STARK for a given problem, we first need to arithmetize it. **Arithmetization** is the reduction of computational problems (which ask whether a string is in a language) to algebraic coding problems (which ask whether a function $f : S \rightarrow \mathbb{F}$ is the evaluation of a polynomial of degree $< |S|/8$). Because polynomials are excellent error-correcting codes (ECC) that add redundancy and spread information, this property implies that small changes don't affect the whole data. Also, by spreading the information, cheats become much more noticeable.

In STARK systems, after arithmetization, the prover needs to convince the verifier that the resulting polynomial from the computational problem has a low degree. This step is done via the FRI protocol (Fast Reed-Solomon Interactive Oracle Proof of Proximity). IOP proofs are those in which the prover sends a function as an oracle, and the verifier has oracle access to it, making queries at chosen points of the function. The idea of FRI is similar to the inverse fast Fourier transform (iFFT) and is done recursively, dividing the polynomial size by 2 at each recursive step. So the verification time is only logarithmic in the size of the computation, achieving the scalability property we mentioned earlier.

Conclusion

We started this survey with definitions for proof systems and their properties. Then we moved from theory to practice, exploring classic protocols like Graph 3-Coloring and Schnorr, and later saw how Fiat-Shamir turns interactive proofs into non-interactive ones, which paved the way for modern systems like STARKs.

We saw that for each protocol, the zero-knowledge property needs to be formally proved, and the soundness error must be reduced as much as possible, since these protocols are used for security-critical applications. We also explored several ways to improve efficiency, making these protocols more practical.

However, even with these efforts, the zero-knowledge property is still achieved under computational assumptions, such as the existence of one-way functions or the random oracle model. Moreover, completeness and soundness are achieved probabilistically, and the error can always be reduced further at the cost of efficiency.

In conclusion, these protocols can still be improved in terms of efficiency and reliability, or they can be studied to inspire even better protocols in the future.

Acknowledgments

I would like to thank Dr. Shahram Khazaei for introducing me to these fundamental references and encouraging me to explore the foundations of zero-knowledge proofs independently.

References

- [1] Berry Schoenmakers, *Lecture Notes on Cryptographic Protocols*, Eindhoven University of Technology, Available at <https://berry.win.tue.nl/CryptographicProtocols/>.
- [2] Oded Goldreich, *Foundations of Cryptography: Volume 1, Basic Tools*, Cambridge University Press, 2001.
- [3] Bar-Ilan Winter School on Cryptography, *Zero-Knowledge Proofs (9th School, February 2019)*, Video lectures and materials available at <https://cyber.biu.ac.il/event/9th-biu-winter-school-on-cryptography/>.